

What my program does

My program's concept is inspired by the old bouncing DVD logo that would change colors as it hit the edge of TV screens (and that are iconically watched to see if the logo would hit the corner of the screen), the overwhelmed feelings of growing up, and learning in a class that the human brain can think a maximum of 3 thoughts at once. It also builds off of the program that Professor Stalbaum presented in lecture, oneball.py. I wanted to build a program that reminisces on the simplicity of childhood, and how that simplicity morphs into worrying and rushing through life as we grow up. It reflects on different ways people think and feel about things like graduation, perfectionism, insecurity, anxiety, managing relationships, and the complex intersection of all these topics in our heads.

The program consists of 3 balls that traverse across the screen and bounce off the edges, just like the old DVD logo. The balls use the same color palette as the DVD logo as well, and similarly changes color whenever it hits the edge of the screen. Referencing the old bouncing DVD logo is meant to call back to our childhoods. Additionally, the balls have text in them that represent the anxious thoughts people worry about in adulthood. The text also changes every time the balls hit the edge. The text's category of thought depends on what color the ball becomes (ex. If the ball becomes yellow, the text relates to a thought about relationships). Additionally, each category had quotes labeled as either having a positive or negative outlook. A random quote is displayed from that category, but if the quote is positive, the ball slows down and becomes smaller. In contrast, if the quote is negative, the ball speeds up and becomes bigger. The intent of this is to express how anxious, negative thoughts can easily take up most of our minds and can make our thoughts feel hectic going 100 miles an hour. On the other hand, positive thoughts help us slow down, but can easily be paid less attention to than worrisome thoughts. Finally, the balls not only hit the edges of the screen, but also collide with one another and bounce off each other. This is meant to visualize how life becomes so fast, and how we think or worry about so many things at once that the thoughts can compete in our head with what we should worry about more. There being 3 balls also represents how we maximize our minds to worry (often about many things at once), especially since humans can only think 3 thoughts at a time.

Database

As mentioned earlier, my project utilizes quotes so that texts are displayed on the balls. I couldn't find a database that had exact quotes on people's anxious thoughts (and it would be a hard thing to research on a big scale). I made my own database by interviewing my peers and asking them what anxious or calming thoughts they felt in

each category. There are 7 colors in the bouncing DVD logo, so I decided on 7 categories for each (each category has a positive and negative counterpart):

- White: perfectionism/indecisiveness vs. reassuring thoughts
- Red: graduation anxiety vs. trusting the process thoughts
- Green: insecurity vs. confidence/aspirations
- Blue: responsibilities vs. resting/celebrating accomplishments
- Cyan: regrets vs. being at peace with the past
- Magenta: optimism vs. pessimism
- Yellow (emphasis on social relationships): conflict/dreading goodbyes vs. making peace with the natural drift in friendships/sentimentality

I gathered the most common general gists of my discussions to summarize the thoughts as different quotes for my database.

For each category, I labelled the quotes as either having a positive or more negative outlook. For every category, there are around 2-3 quotes for each positive and negative labels.

```
quotes_by_color = {
  (255, 255, 255): [
    {"text": "I could be better at...", "mood": "sad"},
    {"text": "What's the perfect decision for...", "mood": "sad"},
    {"text": "I'm trying my best and will improve on...", "mood": "happy"},
    {"text": "This best decision right now for me is...", "mood": "happy"},
  ],
  (255, 0, 0): [
    {"text": "Am I on the right timeline for...", "mood": "sad"},
    {"text": "What will I do after...", "mood": "sad"},
    {"text": "I'll figure it out and enjoy processing...", "mood": "happy"},
    {"text": "It'll be okay even if...", "mood": "happy"},
  ],
  (0, 255, 0): [
    {"text": "There's no point in trying to...", "mood": "sad"},
    {"text": "I won't be capable of becoming a...", "mood": "sad"},
    {"text": "I'm capable of becoming a...", "mood": "happy"},
    {"text": "I want to try...", "mood": "happy"},
  ],
  (0, 0, 255): [
    {"text": "I need to do...", "mood": "sad"},
    {"text": "I must accomplish...", "mood": "sad"},
    {"text": "I can rest because...", "mood": "happy"},
    {"text": "An accomplishment I can celebrate is...", "mood": "happy"},
  ],
  (0, 255, 255): [
    {"text": "I should have...", "mood": "sad"},
    {"text": "I shouldn't have...", "mood": "sad"},
    {"text": "I can't change that...", "mood": "happy"},
    {"text": "I can recover from...", "mood": "happy"},
    {"text": "I understand why I chose to...", "mood": "happy"},
  ],
  (255, 0, 255): [
    {"text": "What if the best will happen...", "mood": "sad"},
    {"text": "What if the worst will happen...", "mood": "happy"},
    {"text": "Was I weird in that interaction...", "mood": "sad"},
    {"text": "We had a great conversation...", "mood": "happy"},
  ],
  (255, 255, 0): [
    {"text": "I'll miss my friends...", "mood": "sad"},
    {"text": "It won't be the same...", "mood": "sad"},
    {"text": "Why'd this conflict happen...", "mood": "sad"},
    {"text": "Who am I without them...", "mood": "sad"},
    {"text": "I'm thankful for these memories...", "mood": "happy"},
    {"text": "We'll see each other again...", "mood": "happy"},
    {"text": "It's okay we disagree...", "mood": "happy"},
    {"text": "It's okay we're drifting...", "mood": "happy"},
  ],
}
```

In my code, I organized this data as a dictionary where the keys were tuples that represented the colors. The values were lists of dictionaries. For each dictionary in that

list, there was a key dedicated to “text” and a key dedicated to “mood”. The value under the “text” key held the text of the quote and the value under the “mood” key held the value of whether it was positive (“happy”) or negative (“sad”).

My code detects whether the ball hits the edge of the screen and if it does, the ball changes to a random color (based on a separate list that holds the color tuples). Then, based on that color, it refers to the dictionary and pulls from the quotes under that color. This is its quote list. From this quote list, it chooses a random quote. Then, the “mood” of the quote is checked. If the “mood” is “sad”, the speed and size alter by increasing both of them. If the “mood” is “happy”, then it’s adjusted oppositely so that the speed is slower and the size is smaller.

My Process

I started out with the [oneball.py](#) program and knew I wanted to work with how it animated the ball to move. I started out experimenting with the program and figured out how to make the ball go in random directions by calculating its angle and having a separate variable for velocity (instead of using the up, down, left, right functions originally given). I had to familiarize myself with the ranks and working across the 4 monitors as I’ve never done that before. I referred to the source code and used it as a template so that I could add my new draw function instead (which considered velocity and putting text on the image). I successfully broadcasted the code for the 1st rank to the other screens. After accomplishing this with one ball, I added the second with troubleshooting in between. I had a hard time adding multiple balls because I originally rendered them (with the draw function) by putting the ball objects in a list and looping through each. This didn’t work though, so I rendered each one separately and had the direction changes for each one separately (so that each ball was kept track of on if it hit the edge) in the “while True” loop at the bottom of the program. The loop tracked if any of the balls hit any of the edges (based on the ball’s x and y coordinates in relation to the universe size across all 4 screens) and if so, it changed the color and chose the quote. Originally, my project during the time for the midterm pulled from a simple list of generic quotes and iterated through it based on an index (that would increase as it went to the next quote then restarted to the beginning of the list). My project at the time of the midterm also didn’t change the speed and size of the ball. I was unfamiliar with displayed text of the screen so I researched and used `pygame.font` to import a font and use built-in functions to render and position the text.

Problems from my project at the time of the midterm included that it didn’t have a specific database, that the text wasn’t very legible, that the text would go outside the circle, that the circles would overlap and hide each other when they crossed one

another, and that the circles would often glitch and get stuck in the wall. I will attach a link to what my project looked like during the midterm here:

<https://drive.google.com/file/d/1fzfPUmNh2lcFbd3357SGP4RnGTCMZChL/view?usp=sharing>

To improve upon this and implement my new ideas after the midterm, I first added a third ball with my preexisting code. I created the new database with my concept of different categories and different “moods” for each. I altered the loop so that it chose category based on color and changed the speed and size based on the “mood” of the quote. I then fixed the legibility by adding a drop shadow behind the text, and rendering that first (the same way as the text). I also created a wrapping text function designed to track if the text exceeded the circle’s width by splitting the words, adding words until it reaches the edge, and going to the next line if that happens. I then edited the draw function to utilize text wrapping so the text would stay inside the circle. I fixed the glitch because I noticed that it’s inner part would go into an edge and stay there stuck and always changing color since it forever detected it was on an edge and couldn’t leave. I edited it so that it sensed which wall it hit and repositioned itself so that the inner part of the circle never went into the wall. Finally, I detected collisions by creating a function that took 2 balls as input and checked if they collided by calculating the distance between them. I made another function where it resolved the collision by reversing the balls’ velocity and making them bounce the other way. I also had an issue here because the balls would glitch on each other similar to how they glitched into the wall. I resolved this by utilizing the ball size and ensuring that the balls didn’t overlap halfway into each other before separating (otherwise, they’d get stuck or move weird). Ball collision is detected in the “while True” loop and if it occurs, the resolving collisions function is called. There was also testing here and there that wasn’t really a bug, but more tuning what would be viewable for the users. This included testing if the ball was moving too fast for the user to read the text, testing font size, etc.